

DSC 180A · PAPER PRESENTATION

Large Margin Classification Using the Perceptron Algorithm

Yoav Freund & Robert E. Schapire · Machine Learning, 1999

Section 2: the perceptron and the voted perceptron

Presented by Andrew

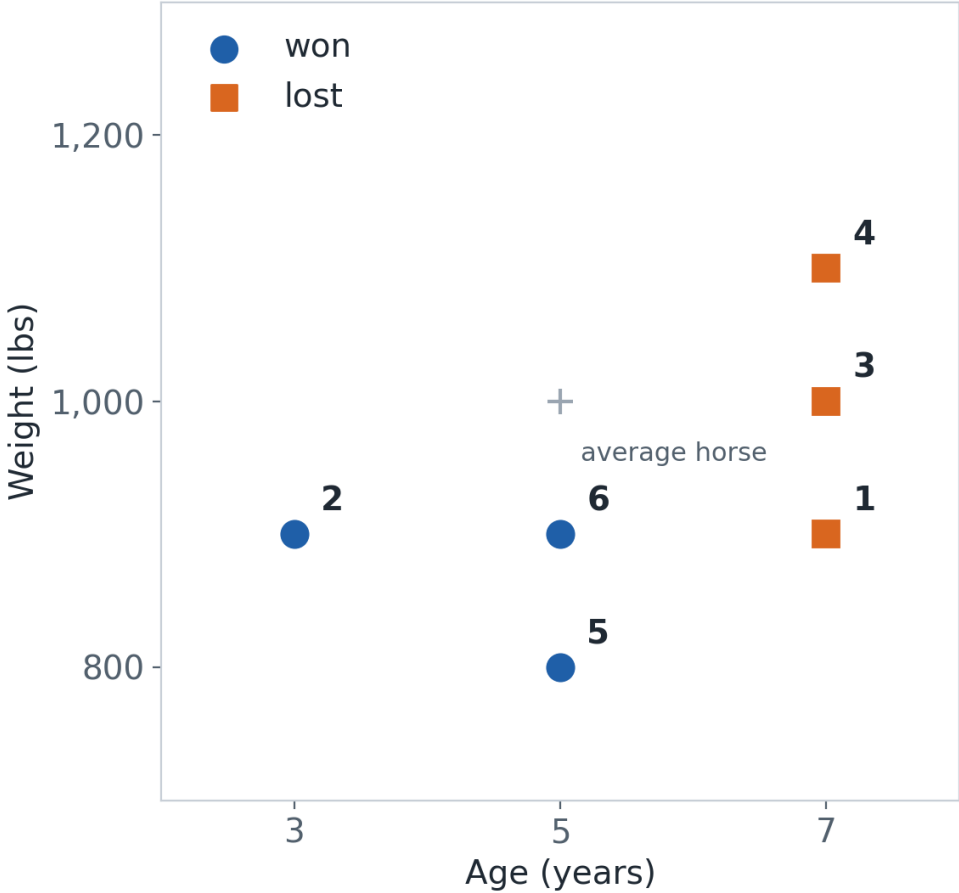
Will this horse win?

#	Horse	Age	Weight	Result	x
1	Iron Duke	7	900	Lost	(2, -1)
2	Little Comet	3	900	Won	(-2, -1)
3	Old Timer	7	1,000	Lost	(2, 0)
4	Big Ben	7	1,100	Lost	(2, 1)
5	Featherweight	5	800	Won	(0, -2)
6	Swift Kid	5	900	Won	(0, -1)

$$x = (age - 5, (weight - 1,000) \div 100)$$

Each horse is measured relative to the average horse (5 years, 1,000 lbs).

Pretend each result is independent, and that age and weight really matter.



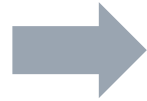
Guess, check, and only change when you're wrong

1 Guess

Will it win? Predict

$\text{sign}(\mathbf{v} \cdot \mathbf{x})$

+ means win

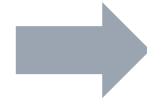


2 Check

Did it win?

$y = +1$ won

$y = -1$ lost



3 Fix only if wrong

$\mathbf{v} \leftarrow \mathbf{v} + \mathbf{y} \cdot \mathbf{x}$

Right? Change nothing.

Then go to the next horse. Keep cycling through all the horses until a full pass has no mistakes.

Reading $\mathbf{v}$: $\mathbf{v} = (\text{age weight}, \text{weight weight})$, one number per feature. A negative number means smaller is better. $\mathbf{v} = (-2, -1)$ means younger and lighter horses win, and age matters twice as much.

Pass 1, horse by horse

Start with $v = (0, 0)$

Horse	x	y	$v \cdot x$	Guess	Right?	v after
1 Iron Duke	$(2, -1)$	-1	0	tie	X	$(-2, 1)$
2 Little Comet	$(-2, -1)$	$+1$	3	win	✓	$(-2, 1)$
3 Old Timer	$(2, 0)$	-1	-4	lose	✓	$(-2, 1)$
4 Big Ben	$(2, 1)$	-1	-3	lose	✓	$(-2, 1)$
5 Featherweight	$(0, -2)$	$+1$	-2	lose	X	$(-2, -1)$
6 Swift Kid	$(0, -1)$	$+1$	1	win	✓	$(-2, -1)$

Update rule

$$v \leftarrow v + y \cdot x$$

Horse 1 lost ($y = -1$)

$$\begin{aligned} &(0, 0) \\ - &(2, -1) \\ = &(-2, 1) \end{aligned}$$

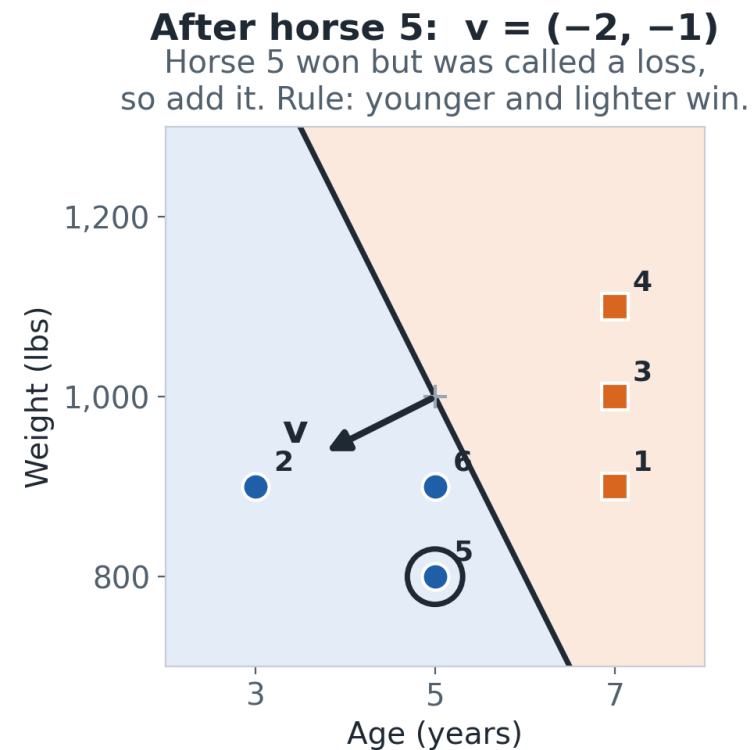
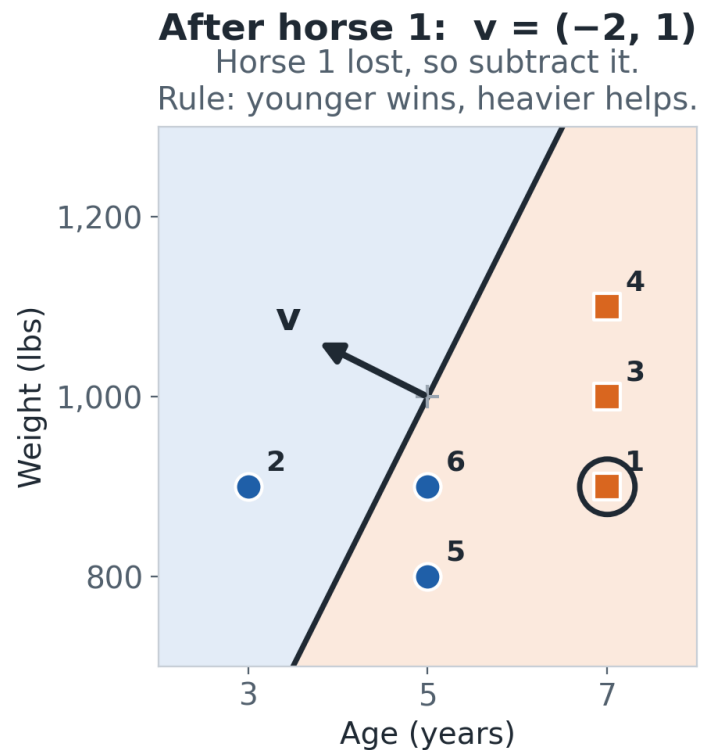
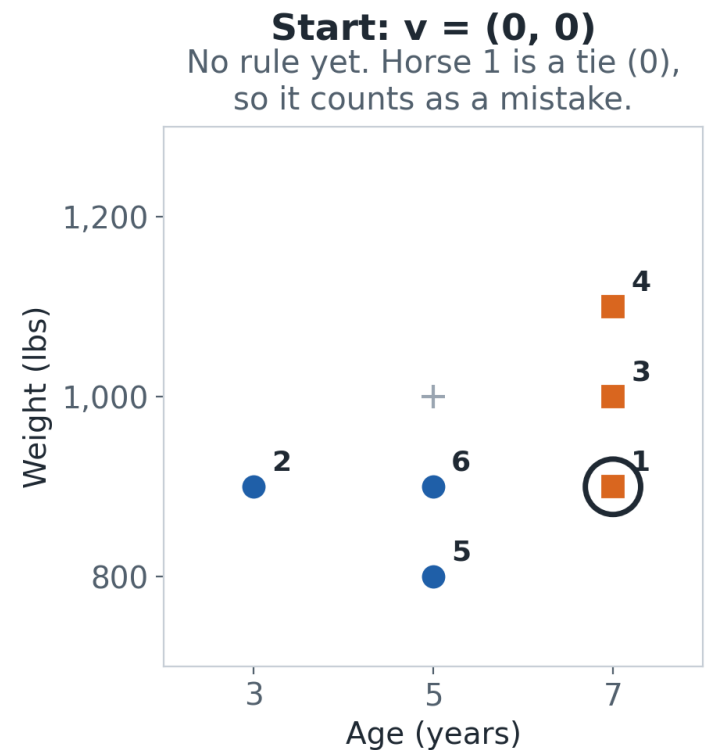
Horse 5 won ($y = +1$)

$$\begin{aligned} &(-2, 1) \\ + &(0, -2) \\ = &(-2, -1) \end{aligned}$$

Pass 2: with $v = (-2, -1)$, all 6 horses are right, so stop.

Orange rows are mistakes. Only those two rows change v .

How v shifts on each mistake



Blue side = predicted win. The circled horse is the mistake that moves the line.

How it runs, and what it's good and bad at



One horse at a time, in order, all the way to the end. Then repeat.

Pros

- Very simple: a few lines of code
- Each step is cheap: one dot product
- Learns as data arrives, nothing to tune

Cons

- Only finishes if a separating line exists
- Can take many passes if classes are close
- Keeps only the last v , with no margin

The classic perceptron's mistakes are bounded

THEOREM 1 (BLOCK, NOVIKOFF) *Let $\langle (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m) \rangle$ be a sequence of labeled examples with $\|\mathbf{x}_i\| \leq R$. Suppose that there exists a vector $\mathbf{u}$ such that $\|\mathbf{u}\| = 1$ and $y_i(\mathbf{u} \cdot \mathbf{x}_i) \geq \gamma$ for all examples in the sequence. Then the number of mistakes made by the online perceptron algorithm on this sequence is at most $(R/\gamma)^2$.*

Proof: Although the proof is well known, we repeat it for completeness.

Let $\mathbf{v}_k$ denote the prediction vector used prior to the k th mistake. Thus, $\mathbf{v}_1 = \mathbf{0}$ and, if the k th mistake occurs on $(\mathbf{x}_i, y_i)$ then $y_i(\mathbf{v}_k \cdot \mathbf{x}_i) \leq 0$ and $\mathbf{v}_{k+1} = \mathbf{v}_k + y_i \mathbf{x}_i$.

We have

$$\mathbf{v}_{k+1} \cdot \mathbf{u} = \mathbf{v}_k \cdot \mathbf{u} + y_i(\mathbf{u} \cdot \mathbf{x}_i) \geq \mathbf{v}_k \cdot \mathbf{u} + \gamma.$$

Therefore, $\mathbf{v}_{k+1} \cdot \mathbf{u} \geq k\gamma$.

Similarly,

$$\|\mathbf{v}_{k+1}\|^2 = \|\mathbf{v}_k\|^2 + 2y_i(\mathbf{v}_k \cdot \mathbf{x}_i) + \|\mathbf{x}_i\|^2 \leq \|\mathbf{v}_k\|^2 + R^2.$$

Therefore, $\|\mathbf{v}_{k+1}\|^2 \leq kR^2$.

Combining, gives

$$\sqrt{k}R \geq \|\mathbf{v}_{k+1}\| \geq \mathbf{v}_{k+1} \cdot \mathbf{u} \geq k\gamma$$

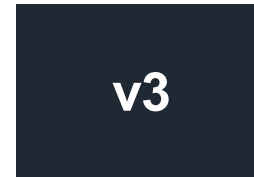
which implies $k \leq (R/\gamma)^2$ proving the theorem. □

mistakes $\leq (R / \gamma)^2$

if some line separates the data

The idea: don't throw the old versions away

Classic perceptron



Keeps only the newest

Voted perceptron

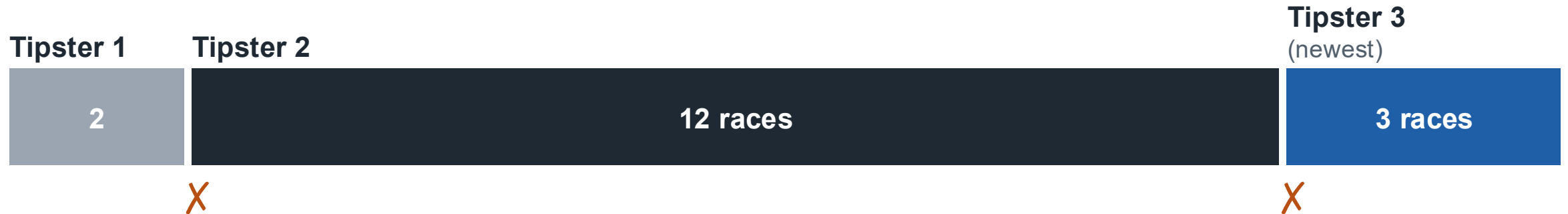


Keeps all of them, and they vote

Same training. The only change is saving every version of v , plus how long it lasted, and letting them vote.

Analogy: which tipster do you trust on race day?

Each block is one tipster's streak of correct calls.



X = a wrong call. That tipster retires, and the next one is the old one plus a correction ($v \leftarrow v + y \cdot x$).

Classic perceptron

Bet only with the newest tipster.

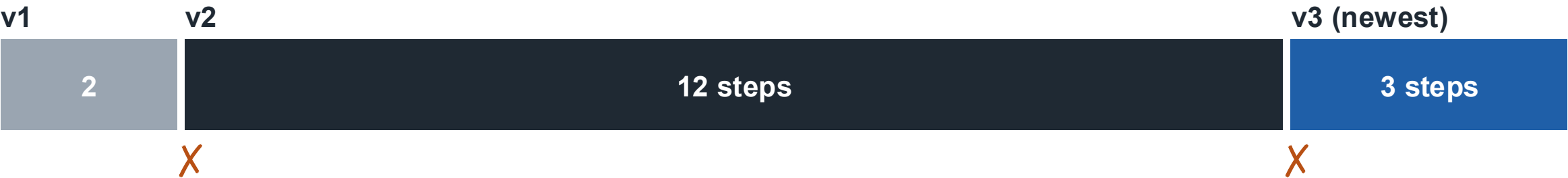
Tipster 3, who has only 3 good calls.

Voted perceptron

Everyone bets. Each vote counts as many times as that tipster's streak.

Votes: $2 + 12 + 3$

Every version votes, weighted by how long it lasted



X = a mistake. That version is replaced by $v + y \cdot x$, and the new one starts counting.

A new horse comes in. Every version makes its call:

Version	Lasted	Says	Vote
v1	2	win	+2
v2	12	lose	-12
v3	3	win	+3
Total			-7

Classic (v3 only): win

Voted (-7): lose v2's long track record wins

The algorithm (Figure 1)

Training

```
v = 0; count = 0
for each horse (x, y):
    if sign(v · x) == y:
        count = count + 1
    else:
        save (v, count)
        v = v + y · x
        count = 1
```

Prediction

```
score =  $\sum \text{count}_k \cdot \text{sign}(v_k \cdot x)$ 
predict sign(score)
```

What's new vs. the perceptron

1 Training

On a mistake, save the old v and its count.

Output of training: a list of pairs

```
<(v1, c1), (v2, c2), ..., (vk, ck)>
ours: <(v1, 2), (v2, 12), (v3, 3)>
```

2 Prediction

Every saved v votes win or lose, times its count.

How it runs, and what it's good and bad at



Weighted vote

Box area = number of votes

Same training as the perceptron. The only change is saving every version and voting.

Pros

Beats the last-vector perceptron (next slide)

Uses every version, not just the newest

One bad update can't swing the answer

Does well even after one pass

Cons

Has to store every version

Prediction asks every version

Still a bit less accurate than an SVM

Vote beats Last, the classic perceptron

Table 3. Results of experiments on individual classes using polynomial kernels with $d = 4$. The rows marked SupVec and Mistake give average number of support vectors and average number of mistakes. All other rows give test error rate in percent for the various methods.

	label	0	1	2	3	4	5	6	7	8	9
$T = 0.1$	Vote	0.7	0.5	1.3	1.5	1.4	1.4	0.9	1.3	1.8	2.1
	Avg. (unnorm)	0.7	0.5	1.3	1.5	1.3	1.3	0.9	1.3	1.8	2.0
	(norm)	0.7	0.5	1.3	1.5	1.4	1.4	0.9	1.3	1.8	2.1
	Last	1.0	0.7	1.7	2.1	1.5	2.8	1.2	1.8	2.4	2.7
	Rand.	2.1	1.3	3.0	3.7	3.0	3.2	2.2	2.7	4.7	4.5
	SupVec	133	89	180	228	179	202	136	160	285	290
	Mistake	133	89	180	228	179	202	136	160	285	290
$T = 1$	Vote	0.3	0.3	0.6	0.5	0.5	0.5	0.5	0.6	0.7	0.9
	Avg. (unnorm)	0.3	0.2	0.6	0.5	0.5	0.5	0.4	0.6	0.7	0.9
	(norm)	0.3	0.2	0.6	0.6	0.5	0.5	0.4	0.6	0.8	1.0
	Last	0.5	0.5	1.0	1.1	0.7	0.8	0.5	1.0	1.2	1.3
	Rand.	0.8	0.6	1.4	1.5	1.2	1.3	0.9	1.2	1.9	2.1
	SupVec	506	407	782	996	734	849	541	738	1,183	1,240
	Mistake	506	407	782	996	734	849	541	738	1,183	1,240

Vote

Every saved version votes ± 1 , weighted by how long it lasted

Avg

Every version's score, weighted by how long it lasted

Last

Only the final version: the classic perceptron

Rand

The version alive at one random step

The experiment: can it spot malignant tumors?

569

tumors (Kaggle / UCI)

30

measurements of the cell nuclei
per tumor

37%

malignant (212), the rest benign

Why this dataset

Close to linearly separable, but not perfectly, which is where voting should help

Small and clean: all numeric, no missing values

Each row is one breast mass, sampled with a needle and measured under a microscope

The paper's questions, on new data

1

Does Vote beat Last?

2

How close is it to an SVM?

3

What does it cost in time?

Setup: malignant = +1, benign = -1. Standardized features, 80/20 train/test split, $d = 1$, $T = 1$ and 3, mean of 50 runs.

Our run: Breast Cancer Wisconsin (d = 1)

	Method	Error % (95% CI)	Train (ms)	Predict (μ s / tumor)	Kernel evals / prediction
T = 1	Vote	3.95 $\pm$ 0.41	1.26	0.3	27
	Avg	3.77 $\pm$ 0.43	1.26	0.3	27
	Last	4.84 $\pm$ 0.66	1.26	0.3	27
	Rand	6.25 $\pm$ 0.59	1.26	0.4	27
T = 3	Vote	3.72 $\pm$ 0.49	2.17	0.7	59
	Avg	3.54 $\pm$ 0.44	2.17	0.6	59
	Last	4.33 $\pm$ 0.55	2.17	0.7	59
	Rand	5.12 $\pm$ 0.44	2.17	0.7	59
SVM	SVC (ref.)	3.95 $\pm$ 1.03	2.38	1.3	34

Vote beats Last (3.95% vs. 4.84% at T = 1, sign test p = 0.017) at no extra prediction cost.

Conclusions

- 1 Weight members by performance
- 2 Learns from streaming data; SVMs retrain
- 3 Ensembles of weak learners help